
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 1: Metamodelling
Project Type: Code Generation
September 25, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Introduction and Scope	1
2	Domain Analysis	1
2.1	Observations From the Existing Configuration	1
2.2	Structural and Operational Concepts	1
2.3	Notation	2
3	Modelling Decisions	2
3.1	Three Model Stages, Two Metamodels	2
3.2	What Each Stage May Contain	2
3.3	Shared Values Are Set by the Platform	2
3.4	Derived Values Cannot Be Overridden	2
3.5	Requirement Names Instead of Target Field Names	2
3.6	Enumerations Instead of Free-Form Strings	3
3.7	Release Grouping and Deployment Order	3
4	Alternatives Considered	3
4.1	A Third Metamodel for the Generated Files	3
4.2	Grammar First or Metamodel First	3
4.3	One Resolved Model, or One Per Application	3
5	The Source Metamodel	4
5.1	The Project Document	4
5.2	The Platform Document	6
5.3	Enumerations	8
6	The Target Metamodel	8
6.1	Required Content	8
6.2	Structure	8
6.3	The Path Plan	10
6.4	The Generated Resources	10
6.5	Keeping Target Field Names Out of the Model	10
7	Implementation in EMF	10
7.1	The Concrete Syntax	10
7.2	The Constraints	11
7.3	How an Invalid Model Is Reported	11
8	Example Models	12
8.1	The Minimal Model	12
8.2	A Resolved Model as a Deployment View	12
8.3	Example Target Models	12

A	Worked Examples	14
A.1	The Minimal Model, End to End	14
A.2	Continuous and Interrupted Applications in One Project	14
A.3	A Rejected Model	14
A.4	The Hand-in Archive	14
B	All Constraints	15

1 Introduction and Scope

Task 0 chose the problem: generate all Kubernetes deployment files for a cluster from one abstract model of what each application needs. For Task 1, two metamodels were built in Ecore: the source language an author writes, and the *resolved model*, which adds every value the platform derives. OCL constraints and an Xtext syntax complete the source language, and example models instantiate both metamodels. Appendix A.4 describes the archive handed in with this report.

The Task 0 feedback led to a smaller scope (Table 1) and to the notation of Section 2.3.

In scope	Out of scope
Two metamodels: the authored source language, and the resolved model.	A metamodel of the generated files. They are text, and are compared as text.
A closed set of six generators for one cluster. Adding a generator requires a recorded decision.	Additional target platforms, and a second cluster.
Validation of a source model, and the constraints that reject an invalid one.	Applying the generated files to a cluster.
Dependency resolution between applications.	Runtime behaviour, scaling decisions and monitoring rules.

Table 1: Scope of the Project After the Task 0 Feedback

Because the generated files are not applied, statements about correctness concern only the generated model and files.

2 Domain Analysis

The domain is the deployment configuration of a small production environment: one Kubernetes cluster of seven nodes, ten namespaces, and about thirty long-running processes in about twenty applications. The configuration is maintained by hand, and each concept below was taken from it.

2.1 Observations From the Existing Configuration

Repeated values. The hostname of one application appears in seven places across three repositories, and two tests exist only to detect when these copies disagree. A port appears in the container specification, the service, the probe and the monitoring rule. Generating the configuration from one model makes such tests unnecessary.

Derivations repeated by hand. Four deployments contain the same rollout block, each written independently. The reasoning is only in comments: a recreate strategy left a period with no running pod, and a JVM cold start needs a much larger startup budget than a static bundle. These are rules about a kind of process, written four times as constants. Such a rule is a *derivation*: it computes a value from the authored model and the cluster’s policies.

Mechanisms instead of requirements. The existing configuration states *mechanisms*, settings of the target syntax: a rollout strategy instead of whether the application may be unavailable during an upgrade, and a node label instead of what the process needs from a machine. A process whose volume cannot be attached twice needs a recreate strategy. Without one, it works on a one-node cluster and fails once a second node exists.

Values shared between applications. A hostname must be unique in the cluster. A graphics card is used by one process at a time. The *edge*, the set of entry points for external traffic, is shared and limited. No application can set these values from its own repository (Section 3.3).

2.2 Structural and Operational Concepts

In the *structural* concepts, a project groups applications, and an application groups the processes released together. A process offers named ports and uses the ports of others. An application is reachable at a hostname whose paths lead to its processes.

Most of the generated output comes from the *operational* concepts: what a process needs from a machine, what its data is worth if lost, and which secrets it may read and how it receives them. They also cover how its health is checked, how long a cold start may take, and whether a new version may run beside the old one during an upgrade.

2.3 Notation

The Task 0 feedback suggested ArchiMate or the C4 model instead of, or beside, UML class diagrams.

A metamodel defines a language's abstract syntax. Ecore implements EMOF, which consists of classes, attributes, references and multiplicities, and a class diagram shows these constructs directly. ArchiMate and C4 describe systems, so neither can define a language.

For the *models*, this report follows the suggestion: Figure 4 draws a resolved model as a deployment view. A resolved model records which process runs on which machine, which application is reachable at which hostname, and which process depends on which. Task 3 could therefore generate an ArchiMate technology view or a C4 deployment diagram from it with a second model-to-text transformation.

3 Modelling Decisions

Seven decisions determine both metamodels.

3.1 Three Model Stages, Two Metamodels

The pipeline has three stages: the authored model, the resolved model and the generated text. Only the first two have metamodels (Section 4.1).

The stages are *not* metalevels: no stage is a type model of the next, so no `instanceOf` relation holds between them. The word metamodel is used only for a language definition.

3.2 What Each Stage May Contain

A *decision* is a value the platform chooses, such as the machines a process may run on. **The authored model contains no mechanisms (Section 2.1), and the generators make no decisions.** A person writes a value only if the platform cannot compute it. The platform records every value it computes in the resolved model, and the generators write these values in the target syntax.

An earlier design with two stages, in which the generator computed values internally, produced three incompatible documents with the same version identifier. With the resolved model, each computed value is recorded in one document, and the authored and resolved documents have different names.

3.3 Shared Values Are Set by the Platform

The platform sets a value if and only if it must be unique in the cluster or uses a shared, limited resource. The application declares every other value, and the transformation copies it unchanged.

Memory and CPU are limited, but if applications could not state them, no process would reserve any. For these two values, the application therefore states its requirement, and the platform checks it against the facts each machine publishes. The check compares a requirement with one machine at a time and does not add up requirements across the cluster, so an overstated requirement is not detected.

3.4 Derived Values Cannot Be Overridden

The platform computes each value it sets from the authored model and the platform document. Each such value has one definition, so the metamodel has no override or exception mechanism (Table 2).

One process in the existing configuration seemed to need an override: its hand-set deadline was much shorter than its neighbours', because it starts in seconds rather than minutes. The process already declares its startup budget, so the fix is a deadline rule that uses the startup budget.

3.5 Requirement Names Instead of Target Field Names

No attribute of either metamodel is named after a field of the generated files (Section 2.1). For example, a process declares the paths it must write to, not a security context. Only the generators contain target field names. If the target metamodel used them, it would only restate the generated objects. A check verifies that no metamodel key is a target field name.

3.6 Enumerations Instead of Free-Form Strings

Where an attribute has a fixed set of values, the metamodel declares an enumeration. A constraint can cover every literal of an enumeration, but not every value of a string. Table 2 gives the observation from the existing configuration.

3.7 Release Grouping and Deployment Order

Two groupings are needed: which processes are released together, and which objects are applied before which. The first is *structural*: the processes of one application switch to the new version together or not at all. The second is *derived* from the dependency graph. Deriving the first from the graph was tried and rejected: the transitive closure of the dependency edges joins all applications in the cluster into one release unit.

4 Alternatives Considered

Table 2 lists the rejected alternatives. Three need more detail.

Alternative	Why it was rejected
Template the target files directly (Helm, Kustomize overlays)	There is no abstract model, so no constraint can be written, no value is derived in one place, and the repeated values remain.
Annotate the target files with extra fields	Tried in the existing configuration: the authored and resolved forms had the same name, so no term could state which document was wrong.
Two stages, with the generator resolving values internally	The generator makes decisions that appear in no model and cannot be reviewed.
A third metamodel for the generated files	The target syntax already has a schema, which a third metamodel would restate.
A grammar first, with the metamodel inferred	The concrete syntax would define the concepts, but the language must be readable from two syntaxes.
Free-form strings instead of enumerations	Tried in the existing configuration: the one free-form field was wrong in every application.
An override mechanism for derived values	A wrong derivation stays hidden behind a per-process reason instead of showing for every process of that kind.

Table 2: Alternatives Considered and Why Each Was Rejected

4.1 A Third Metamodel for the Generated Files

A third metamodel would let the generated output be checked structurally instead of as text. As a hand-maintained copy of the target schema, however, it could drift, and it would add no constraint beyond those of the second metamodel. The generated files are therefore compared as text with reviewed expected output, and validated against their own schemas. The typed target resources are classes in the second metamodel (Section 6.4).

4.2 Grammar First or Metamodel First

The source language has a textual syntax, so a grammar could have defined it, with the metamodel inferred. The production implementation, however, parses a different serialisation of the same language. If a grammar defined the language, the two syntaxes would agree only by chance. With the metamodel first, both are checked against one structure, which is exported as a machine-readable description for testing both implementations.

4.3 One Resolved Model, or One Per Application

The resolved model covers all applications in one document, because some values cannot be computed for one application alone. Examples are which edge entry point serves a hostname, the order in which groups of objects are applied, and what an application derives from the applications that depend on *it*. A per-application model would compute these inconsistently or omit them.

Each application's repository receives a *projection*: the part of the cluster-wide model that belongs to that application. It is obtained by filtering rather than by a separate computation, so the two cannot disagree.

5 The Source Metamodel

The source metamodel, `metamodels/project-intent.ecore` in the archive, defines two kinds of document. A *project* document, written by the owner of a group of applications, states what those applications need. One *platform* document states what the cluster offers, including every shared value (Section 3.3).

5.1 The Project Document

An application declares a migration if and only if it uses a database, and each project's one database is migrated by one application. The abstract class `Migration` has three subclasses. The platform's runner applies the changelog of a `ManagedMigration` before any new version starts. A `SelfMigration`, which a third-party image needs, leaves the schema to the image. A `NoMigration` leaves it to another application.

A process's `cutover` states whether it must keep serving during an upgrade. With `continuous`, the new version starts beside the old one and receives traffic only after every process of the application passes its metric checks. Such a switch is called *blue/green*. With `interrupted`, the old version stops first. A process's `lifecycle` is `application`, `job` or `prepare`. A *prepare process* is irreversible one-off setup, such as a seed, that runs after the migration and before the new version starts, and serves no traffic.

An exposure belongs to the application, because one hostname serves several processes, selected by path. A route and a dependency both refer to a `Surface` by name: a route to a process of its own application, and a dependency to a process of another application.

Class	What it represents
Project	The authored document: an owner and the applications it groups.
Application	A unit released as a whole, with an identifier unique in the cluster.
Process	One containerised process: image, kind, ports, needs and health.
Surface	A named port a process offers, declared once.
Exposure	A hostname the application is reachable at, and who may reach it.
Route	A path on that hostname, and the process and surface that serve it.
Placement	What a process needs from a machine, as hard requirements.
Volume	Storage a process uses, with its durability class: what its data is worth if lost.
Probe	The endpoint that serves a process's readiness or liveness check.
DependencyEdge	A link to another application's named surface. Where the provider has databases, it states how the derived database credential may rotate.
Credentials	The rotation an application supports for the database credential derived for its dependency.
Migration	How the schema is migrated: by a Liquibase changelog the platform applies, by the image itself, or not at all.
Grant	A secret a process may read, how it is delivered, and its rotation.
Asset	A file a process needs that is configuration rather than code.
Observability	Where metrics are collected, and the alert level for failures.
Sidcar	A helper process sharing the main process's lifetime.
Capacity	An instance count above one, with the reason it is not derived.

Table 3: Classes of the Project Document

Figure 1 shows the project document, and Table 3 describes its classes.

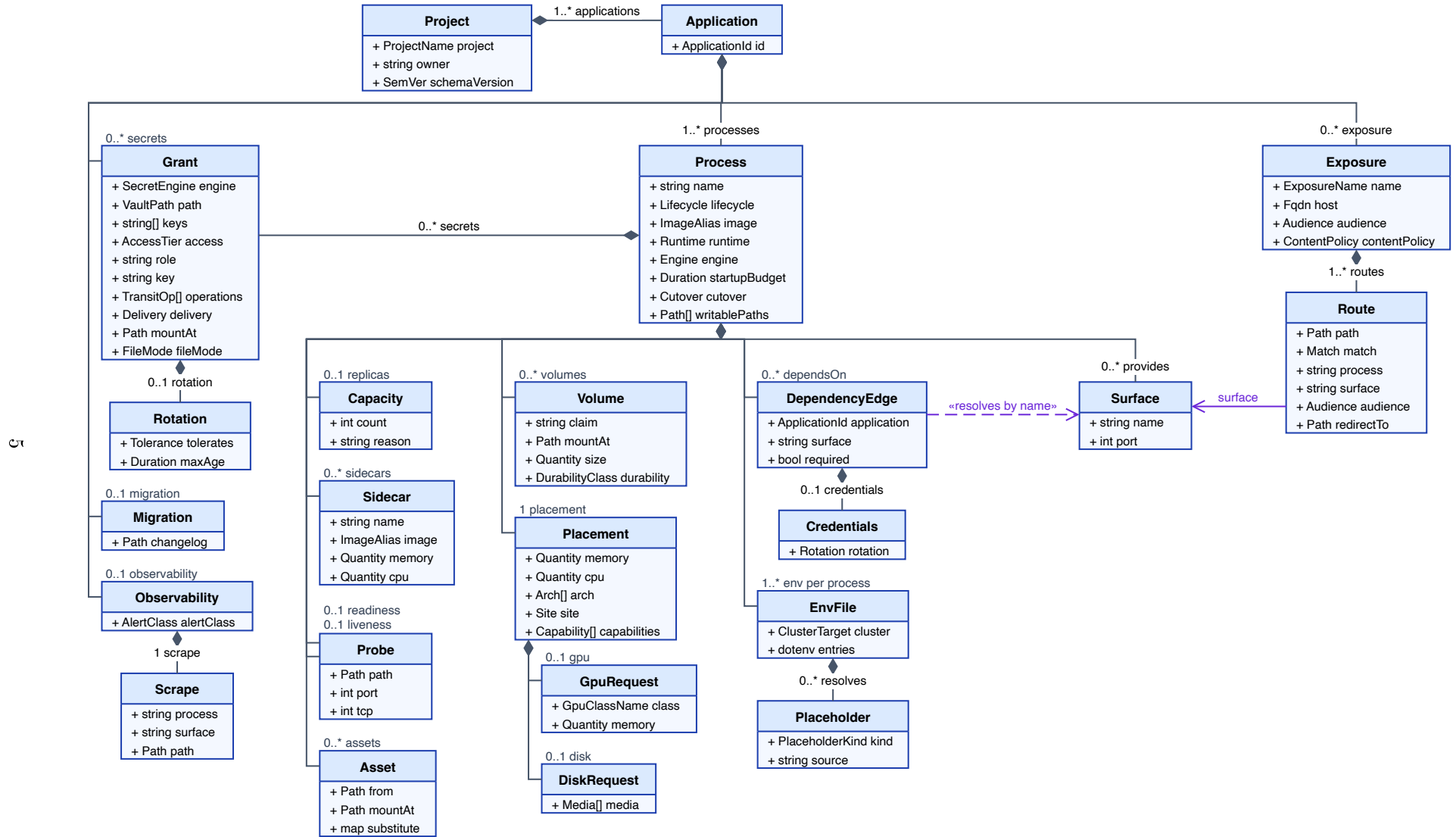


Figure 1: The Project Document

5.2 The Platform Document

The platform document holds the facts and policies that no single application may set. Each platform value describes a fact, such as whether the cluster encrypts secrets at rest, not the setting that implements it (Section 3.5). Two platform values refer by name to applications that a project document declares: a tier's proxy and the delivery machinery. Constraints use two facts. Without encryption at rest, every secret delivery is rejected. A volume or process whose durability class or engine has no policy is rejected (Section 7).

Class	What it represents
Platform	The authored document, one per cluster, with its owner and its hardening level, the security restrictions every container runs under.
PlatformMetadata	Which cluster and project the document describes, and the digest of the machine contract (the facts each machine publishes) it was written against.
Substrate	Cluster facts that other decisions depend on: its version, datastore, server count, whether secrets are encrypted at rest, and its network plugin and policy controller.
Bootstrap	What must exist before the first generated object can be applied. The document records it, and the generators do not create it.
FluxSource	The source reference of Flux, the tool that fetches the generated files and applies them to the cluster.
RenderedArtifacts	The registry repository each project's generated output is published to, one signed artifact per project.
ArtifactSigner	The keyless identity, an issuer and a subject, whose signature Flux accepts on a published artifact.
VaultState	Whether the secret store is unsealed, the second bootstrap fact.
Tier	An entry point of the edge: its audiences, listener, certificate source, authentication endpoint, and the declared Application that acts as its proxy.
DurabilityPolicies	The three durability classes' policies, held together so a missing one is visible.
DurabilityPolicy	One class's backup window and retention.
OffClusterCopy	Where an irreplaceable volume's copy goes, and the credential that writes it.
EnginePolicies	The backup method per engine, held together for the same reason.
EnginePolicy	One engine's backup method: the alias of an image that performs the backup. The document contains no command or script.
MonitorCadence	How often metrics are collected, and the collection timeout.
ProbeCadence	The cluster's probe timing: period, timeout and the number of failures tolerated.
EphemeralPolicy	The size a declared writable path receives.
Provider	A service the cluster runs that this model does not deploy, with its address and named surfaces, which a dependency may refer to.
MigrationPolicy	The runner image alias for every managed migration, with the deadline and resources of a run.
DeliveryPolicy	The <i>delivery machinery</i> : the applications that switch other applications to a new version and are not switched themselves.
AnalysisPolicy	How often a new version is checked, how many passing checks promote it, and how many failing ones roll it back.
HandoverLedger	Which delivery path each project uses while the old path is phased out: the projects still on it, the projects delivered from their pinned artifact, and the date the old path is removed.

Table 4: Classes of the Platform Document

Figure 2 shows the platform document, and Table 4 describes its classes.

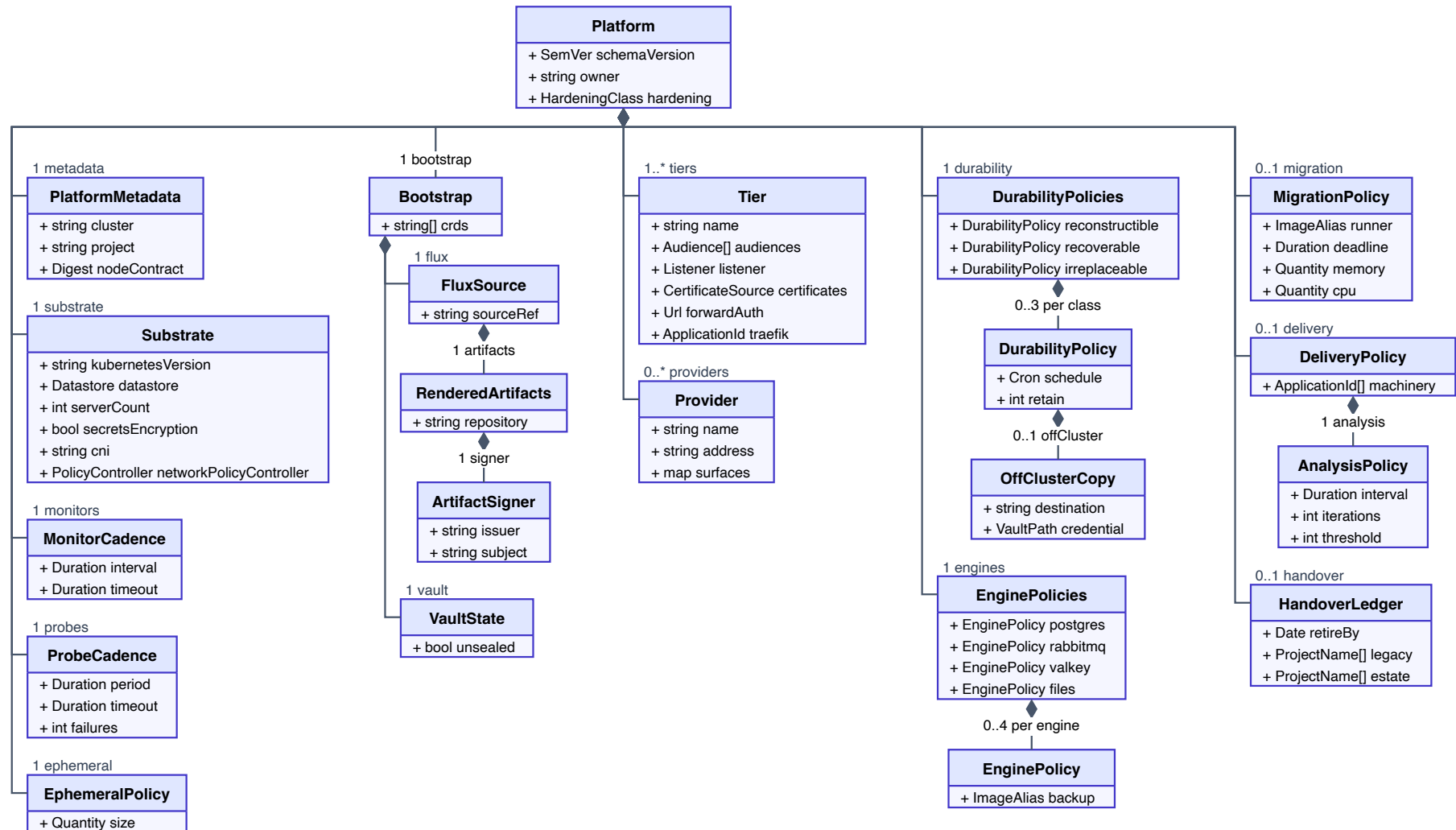


Figure 2: The Platform Document

5.3 Enumerations

Each attribute with a fixed set of values has an enumeration type. In Figures 1 and 2, it is the attribute's type, such as `Cutover`, `DurabilityClass` or `Listener`. Most constraints test these enumerations (Section 3.6).

6 The Target Metamodel

The target metamodel, `metamodels/resolved-deployment.ecore` in the archive, is the output language of the Task 2 model-to-model transformation and the input language of the Task 3 code generation. A target model records the platform's decisions in the terms of the source metamodel, with the typed target resources and their output paths.

6.1 Required Content

The generators use only the resolved model, so it records every value a generator would otherwise decide (Section 3.2).

Decisions. The model records every value the platform derives for an application, a hostname, a process and a volume. Examples are a hostname's edge tier and middleware chain, a process's eligible machines, identity, image digest and probe timings, a volume's backup plan, and the network-policy peers a dependency allows. Each process's *switchover*, the way its new version replaces the old, is derived from its *cutover*: `blue-green`, `stop-start`, or `rolling` for the delivery machinery.

Order and atomicity. The model records the *reconcile units*, the groups of objects that Flux applies together, and the edges that order them. For each blue/green application it records the inputs of its *release gate*, the controller that determines when a new version may receive traffic: the members, their readiness and metric checks, the check interval and the deadline.

Identity of a release. Each application's *revision* is the digest of its element in the resolved model, excluding the revision itself. It changes only when a decision about that application changes, so a migration, a release gate and a release held after a failure can all refer to one release of one application.

Provenance. The model records a hash that identifies the generation run, and one digest per input: each project document, the platform document, the machine contract, the image lock (the pinned image digests) and a snapshot of observed cluster state. Identical digests must produce identical output, so any difference is a defect in the generator.

6.2 Structure

Figure 3 shows the metamodel. The root class, `ResolvedDeployment`, covers all applications (Section 4.3). It contains the provenance, the resolved applications, the reconcile units and the path plan.

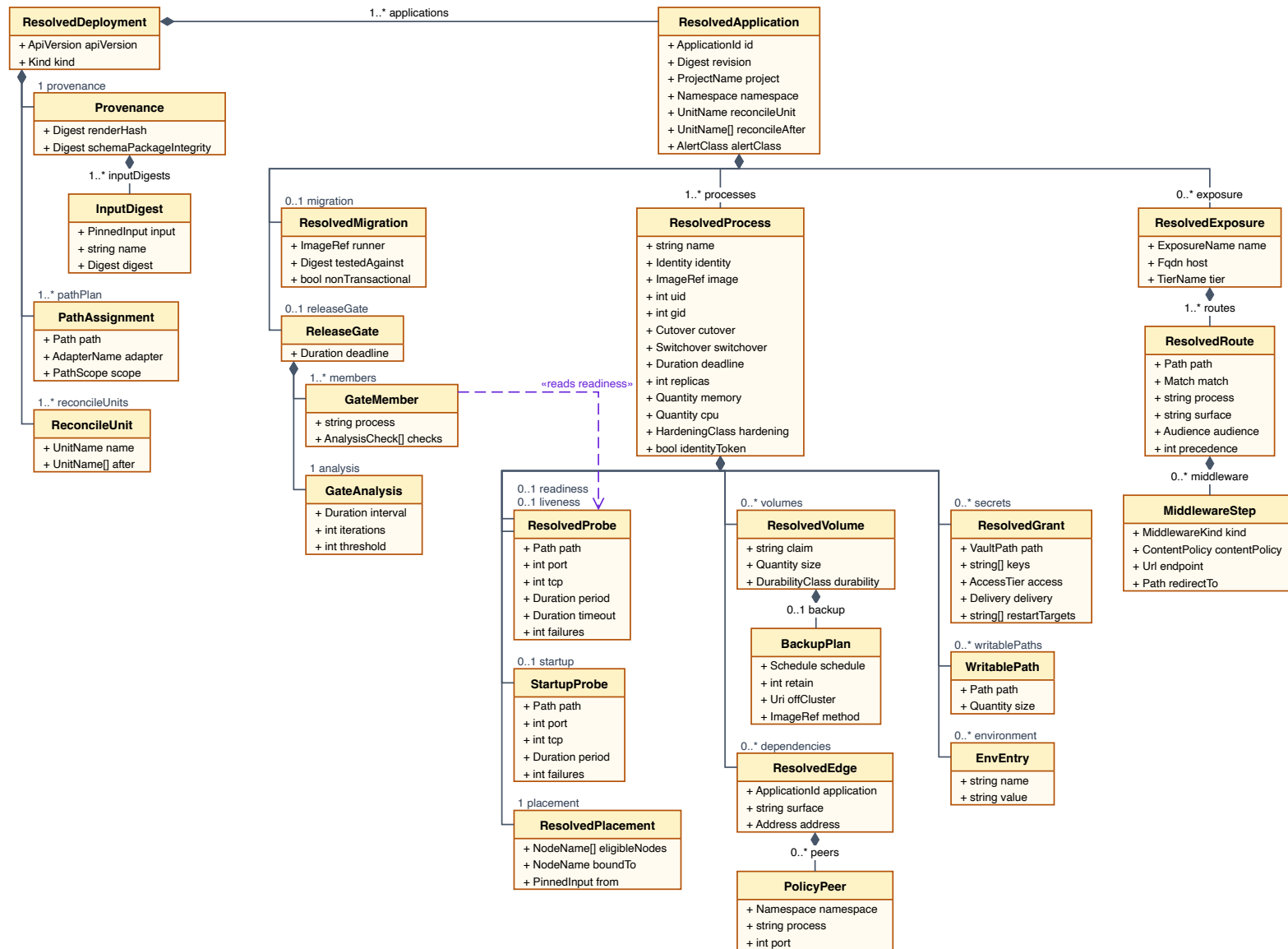


Figure 3: The Target Metamodel

A **ResolvedApplication** is both an element of the cluster-wide model and the projection published to the application's repository. Only the published projection has its own provenance.

A **ReleaseGate** is absent when no process switches blue/green, because an **interrupted** application stops before the new version starts and the delivery machinery updates in place. A **ResolvedMigration** records whether the release contains a changeset that cannot run in a transaction, which no automatic undo may touch. It does not record the migration's identity, owner role and deadline, because these follow from the application and the platform document.

The **cutover** and **switchover** of a **ResolvedProcess** are unset for a prepare process, which replaces no running version.

6.3 The Path Plan

The *path plan* records, for each object to be generated, the responsible generator and the output path, so no generator chooses a path (Section 3.2). Paths chosen per generator failed in two observed cases. An object belonging to a group of applications was emitted once per application, at several paths. A cluster-wide object landed in whichever namespace its generator used. With the paths in the model, the check that no two generators write one path runs before any generator starts.

6.4 The Generated Resources

Generation uses a closed set of six generators, one per subsystem (Table 1). They produce workloads and their supporting objects, network policy, monitoring, edge routing, secret-store policies, and secret delivery. The supporting objects include the *Canary*, the object through which Flagger switches a blue/green process. Flagger is the tool that moves traffic from an old version to a new one.

The metamodel for the code. The generated code is a set of Kubernetes, Flux, Flagger, Traefik and Vault files. The decision classes record *what* each file contains. One typed class per resource family records *which* file is generated, at which path, by which generator: **WorkloadFile**, **SwitchoverFile**, **NetworkPolicyFile**, **MonitorFile**, **IngressRouteFile**, **SecretFile**, **IdentityFile**, **NamespaceFile** and **IndexFile**. A model-to-text template therefore writes one file per typed resource, without making decisions.

6.5 Keeping Target Field Names Out of the Model

Under the naming rule of Section 3.5, the resolved model records that a process switches blue/green, and its typed resource is a **SwitchoverFile**. Only the generator writes it as a *Canary*. The model records the hardening level and writable paths instead of a security context, and the middleware chain as named steps instead of the proxy's syntax. A check verifies that each generator's target vocabulary appears only in that generator.

7 Implementation in EMF

Both metamodels are implemented in three Eclipse bundles under **emf/**, built by Maven and Tycho on JDK 21. The **metamodel** bundle holds the two **.ecore** files, the **.genmodel** files from which the build generates the typed Java, and the Complete OCL file. The **syntax** bundle holds the two Xtext grammars and the generated editor. The **cli** bundle holds the *pipeline*, the program that takes authored files and writes either the parsed model or the diagnostics that reject it. **./mvnw verify** builds and tests all bundles without an Eclipse IDE, and the projects can also be imported into Eclipse Modeling Tools. A test suite runs every example through the pipeline and compares the output with committed expected files.

7.1 The Concrete Syntax

Two Xtext grammars define the concrete syntax, one per authored document (Section 4.2). Both import the registered metamodel package and return its classes. No rule declares its own type, so every concept in the syntax exists in the metamodel. The pipeline chooses the grammar by file name: **platform.intent.yml** is a platform document, and ***.project.yml** a project document.

The syntax is the subset of YAML that the existing configuration files use, so current tools can still read the authored documents.

Indentation as tokens. A step before the parser turns indentation into synthetic **BEGIN** and **END** tokens, so an indented block, a flow mapping and a flow sequence all parse through one rule instead of three.

Unordered keys. Each rule is an unordered group, because key order in a mapping has no meaning. Rules written as sequences would reject documents the metamodel accepts.

Names as cross-references. Where a route refers to its process and surface, the grammar declares a cross-reference, and the linker resolves the name. An unresolved name, such as an undeclared process in a metrics scrape (`E_UNKNOWN_PROCESS`), is therefore a linking error, not an invariant violation. The handover ledger’s project names are plain strings, checked by an invariant, because a project is a whole document and not an object that can be linked to.

7.2 The Constraints

Thirty-six invariants are written in Complete OCL against the source metamodel, with twenty-eight distinct diagnostic codes. Three codes appear on more than one context class: the code for a restated shared declaration on seven classes, and two other codes on two classes each.

The OCL document follows one naming rule: **an invariant is named after the code it reports, and its context is the class of the object the diagnostic points to**. A violation is therefore a (code, path) pair that needs no lookup table, where the path is the RFC 6901 JSON Pointer of the rejected object. Table 5 shows one constraint for each scope, and Appendix B lists all twenty-eight codes.

Code	Context	What it rejects
E_CUTOVER_UNHONOURABLE	Process	One object: a continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	One document: an application whose processes have different cutover values, although it switches as one unit.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	The platform document: a tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	Across documents: an audience no edge tier in the platform document serves.

Table 5: One Constraint for Each Scope

Ten constraints have a project-document context class but use facts from another document: the platform document, or, for the three migration codes that check whether a provider has a database, the provider’s project file. Each invariant is placed on the object whose author must fix it, such as a volume that requests a guarantee the cluster does not provide. These constraints reach the other document through `allInstances()`, whose extent is the resource set the document was loaded into. A guard makes each hold trivially when a project file is loaded alone (Example 1).

```
context Exposure
  inv E_NO_TIER_FOR_AUDIENCE:
    Tier.allInstances()->notEmpty() implies
      Tier.allInstances()->exists(audiences->includes(self.audience))
```

Example 1: `E_NO_TIER_FOR_AUDIENCE`, with a guard for a project file loaded alone

EMF reads an unset enumeration attribute as its first literal, so an unset optional enumeration looks the same as one set to its first value. Each optional enumeration therefore has its own literal for absence, **absent**, which a document cannot write, and the invariants test against **absent** instead of **null**.

Constraints that need all project documents at once, such as two applications with the same identifier, are checked when the documents are combined, not in the OCL document.

7.3 How an Invalid Model Is Reported

The pipeline reports each error as a diagnostic with four fields: the code, the file name, the JSON Pointer of the rejected object, and a message (Appendix A.3). The pipeline checks the documents in three steps and stops after the first step that finds an error.

First, the pipeline parses each file. A syntax error gets the code `schema`, with the line number in its message.

Second, the pipeline registers the Complete OCL document as the EMF validator of the metamodel package and runs EMF’s `Diagnostician` on each parsed document. The code of each violation is the invariant’s name,

read from the Eclipse OCL message. The path is built from the rejected object’s containment chain: the name of each containing feature, with the index for a many-valued feature. All violations are reported. A name the linker could not resolve gets its `E_UNKNOWN_` code at the pointer of the object that holds it.

Third, the pipeline loads the files of a set into one resource set and checks them again. The cross-document invariants held trivially when each file was checked alone, so this step reports only errors the documents cause together, such as a tier whose proxy no project file declares.

A rejected run writes `diagnostics.json` and exits with code 1. An accepted file is written as `intent.json` and `intent.xmi`, and the run exits with code 0. The generated Xtext editor runs the same OCL validator and marks each violation on its source line, with the code as the message.

For each code, the test suite also runs a committed model from `refusals/` that violates it, and compares the output with that model’s expected diagnostics. A constraint tested only on valid models might never report an error. The production implementation declares the same language by hand and is tested against the same committed example files. Later tasks cover that comparison.

8 Example Models

Five example models instantiate the source metamodel, each testing a different part of it (Table 6). Appendix A.4 gives their paths, and Section 7 describes the rejected models.

Example	What it tests
minimal	The smallest complete model: one project, one application, one process, and only required values.
auth	Two continuous processes in one application, so the release gate has two members and its deadline is the maximum over the members. It also has a managed migration, because the application uses a database.
data	Several applications in one project share a namespace. Each volume’s durability class determines its backup plan.
knowledge	Two applications in one project, because their cutover values differ: a continuous API that migrates the project’s schema with a changelog, and an interrupted worker with a volume and no migration of its own. It also has dependencies between applications, secrets shared at project level, and an exposure whose routes have different audiences.
delivery	Two applications of the delivery machinery: Flagger and the release gate. Their processes are continuous but are updated in place instead of switched blue/green, because the release gate cannot control its own release.

Table 6: The Example Models and What Each Tests

Each example has three stages: the authored project file, the resolved model, and the generated file tree. Appendix A follows three examples through these stages.

8.1 The Minimal Model

The minimal model is considerably shorter than the current hand-written file for the same deployment. It declares no rollout strategy, resource limits, security context, labels, selector, namespace, probe timings or instance count. Each of these is derived from a declared value, set by platform policy, or rejected, so the model shows the minimum an author has to write.

8.2 A Resolved Model as a Deployment View

Figure 4 shows the **knowledge** example’s resolved model as a deployment diagram: the project’s two processes on two machines, the volume one of them is bound to, the edge tier and hostname in front, and the two platform services the processes depend on. Each box and arrow is taken from the resolved model (Section 2.3).

8.3 Example Target Models

For the minimal example, the resolved model that the Task 2 transformation must produce was written and reviewed by hand, as `minimal.resolveddeployment`. It is the expected output of that transformation and the first input for the Task 3 templates, so both tasks can be tested before either is complete.

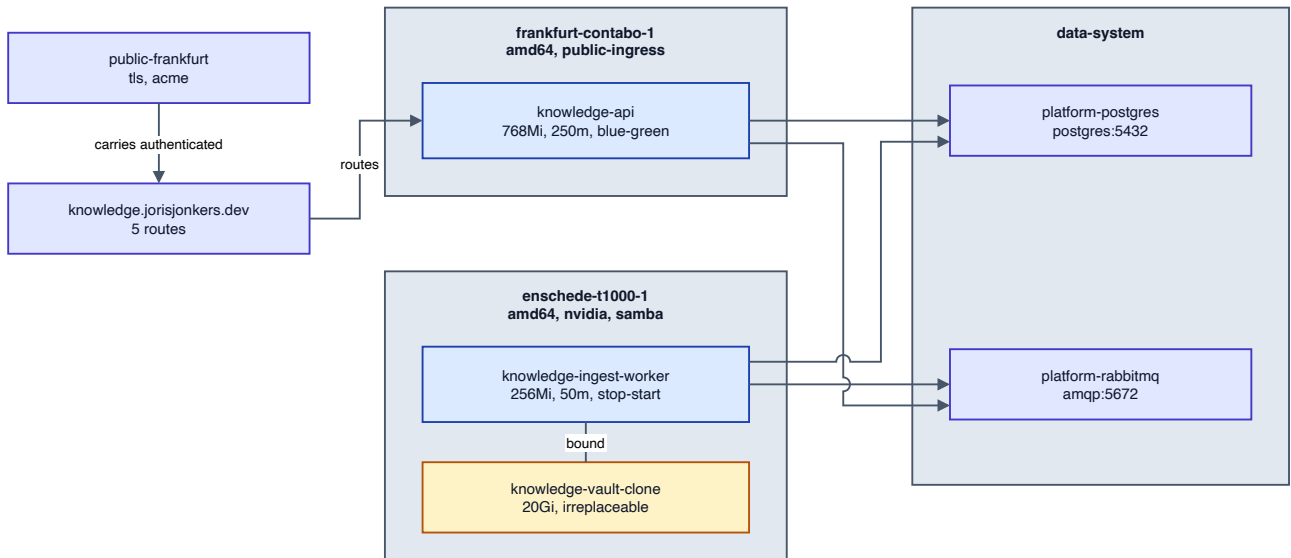


Figure 4: The knowledge Example's Resolved Model as a Deployment View

A Worked Examples

This appendix names the files to read for three examples, in the archive layout of Appendix A.4.

A.1 The Minimal Model, End to End

The input is `examples/minimal/1-input/notes.project.yml`. EMF does not write attributes that hold their default value, so `lifecycle` and `alertClass` are missing from `intent.xmi` beside it. Table 7 compares what the author wrote with what the hand-written resolved model derives (Section 8). The output in `3-output/` has nine files. In `apps/notes/canary.yaml`, the Canary that switches the process, every value comes from the resolved model.

Authored	Derived in the resolved model
<code>project: notes</code> <code>cutover: continuous</code>	<code>namespace="notes-system"</code> , and the reconcile unit (Section 6) <code>switchover="blue-green"</code> , and a <code>releaseGate</code> with this process as its one member
<code>startupBudget: 20s</code> <code>runtime: node</code>	a startup probe of 4 checks 5 seconds apart, and a deadline of 60 seconds the checks <code>error-rate</code> and <code>latency</code> , because the profile exposes HTTP server metrics
<code>image: notes-api</code> <code>audience: anonymous</code>	the image digest, and the numeric user and group it runs as the edge tier <code>public-frankfurt</code> , and a middleware chain without an authentication check
<code>placement</code>	the machines the process is eligible for: all seven, because nothing narrows the set

Table 7: The Minimal Model: Authored and Derived Values

A.2 Continuous and Interrupted Applications in One Project

The input is `examples/knowledge/1-input/knowledge.project.yml`, with the two applications of Table 6. In `2-resolved/`, the continuous application (`resolved.json`) has a release gate and a migration. The interrupted one (`resolved.knowledge-ingest.json`) has neither (Section 6). In `3-output/apps/knowledge/migration.y` the migration becomes two Jobs named after the revision: one that migrates the schema, and one that would undo it. Both are generated suspended, so only the release gate can start them.

A.3 A Rejected Model

`refusals/prepare-process-serves.project.yml` declares a prepare process with a port, probes, replicas and a `cutover`, which only a serving process has. Its expected diagnostics, `prepare-process-serves.diagnostics.json`, hold one entry: the code `E_PREPARE_PROCESS_SERVES`, the document, and the path `/applications/0/processes/1`.

A.4 The Hand-in Archive

The archive handed in with this report contains the report, the metamodels and every model mentioned here, copied from the repository by a script. Table 8 lists its layout. Each XMI or JSON file has a YAML view beside it, converted as the archive's `README.md` describes. A resolved model exists as XMI for `minimal`, and as JSON projections from the production implementation for `minimal`, `knowledge` and `delivery`. The Task 2 transformation will produce the others.

Path	What it holds
<code>metamodels/</code>	<code>project-intent.ecore</code> , <code>resolved-deployment.ecore</code> , and <code>project-intent.ocl</code> , the constraints of Section 7
<code>examples/<name>/1-input/</code>	the authored files in concrete syntax, <code>intent.xmi</code> (the same model as an instance of the source metamodel) and <code>intent.json</code> (its parsed form)
<code>examples/<name>/2-resolved/</code>	the resolved model: <code>minimal.resolveddeployment</code> as XMI, and <code>resolved*.json</code> projections
<code>examples/<name>/3-output/</code>	the generated file tree
<code>examples/platform/</code>	the Platform document, and its model as XMI
<code>refusals/</code>	every rejected model with the diagnostics it must produce, listed with its code in <code>README.md</code> ; a directory is a set of documents read together

Table 8: Layout of the Hand-in Archive

B All Constraints

Table 9 lists each invariant of Section 7 by the code it reports, with its context class.

Code	Context	What it rejects
E_ALERT_CLASS_WITHOUT_SIGNAL	Observability	An alert level is set, but no metrics are collected.
E_CUTOVER_UNHONOURABLE	Process	A continuous process whose storage cannot attach to the second copy a blue/green switch starts.
E_CUTOVER_MISSING	Process	A process with no cutover value at any level. A prepare process is exempt, because it replaces no running version.
E_RELEASE_UNIT_MIXED_CUTOVER	Application	An application whose processes have different cutover values, although it switches as one unit.
E_PREPARE_PROCESS_SERVES	Process	A prepare process with a port, probes, replicas or a cutover , which only a serving process has.
E_SHARED_DECLARATION_DUPLICATED	seven classes	A shared declaration repeated unchanged at a lower level, where it has no effect.
E_SHARED_QUANTITY	Placement	A memory or CPU quantity declared above process level, where it would mean something different for each process.
E_PLACEMENT_INCOMPLETE	Process	A process with no memory or CPU quantity of its own. No higher level may supply one.
E_OWNER_ROLE_GRANTED	DatabaseGrant	A hand-written grant of the database role that changes the schema. The platform derives this role for the migration only.
E_MIGRATION_OWNER_DUPLICATED	Migration	A second application in one project that declares a migration for the project's database.
E_ENGINE_WITHOUT_DURABILITY	Process	A backup engine declared where no volume's durability class requires a backup.
E_DURABILITY_WITHOUT_ENGINE	Volume	A volume that requires a backup, with no engine to state the backup method.
E_ENV_CANNOT_RELOAD	Rotation	A secret that tolerates a reload but is delivered as an environment variable, which is read only at start.
E_ILLEGAL_DELIVERY_FOR_ACCESS	KvGrant	custody delivered as an environment variable or file, although its paths are only created at runtime, or self-renew delivered as an environment variable, although its identity cannot read the path.
E_NON_KV_DELIVERY	TransitGrant, DatabaseGrant	A transit or database grant delivered as an environment variable or file. A transit key never leaves the secret store, and a database credential is issued per lease, so the application fetches both itself.
E_DUPLICATE_ROUTE_MATCH	Route	Two routes of one exposure with the same path and match type, which the derived precedence cannot order.
E_NO_FORWARD_AUTH_ENDPOINT	Tier	A tier that serves authenticated visitors but declares no endpoint to check whether a visitor may enter.
E_HANDOVER_BOTH_PATHS	HandoverLedger	A project listed on both delivery paths, so two sources would both apply it.
E_NO_TIER_FOR_AUDIENCE	Exposure, Route	An audience no declared edge tier serves, so the route reaches no edge.
E_NO_ENGINE_POLICY	Process	An engine with no backup method in the platform document.
E_NO_DURABILITY_POLICY	Volume	A durability class with no policy in the platform document.
E_SECRETS_AT_REST_REQUIRED	Grant	A secret written into a cluster that does not encrypt secrets at rest.
E_MIGRATION_UNDECLARED	Application	An application that uses a database but declares no migration.
E_MIGRATION_WITHOUT_DATABASE	Migration	A migration on an application that uses no database, checked only when the documents of all its providers are loaded.
E_CREDENTIALS_WITHOUT_DATABASE	Credentials	A rotation override on a dependency whose provider has no database.
E_NO_MIGRATION_POLICY	Migration	A changelog, where the platform document defines no migration runner to build it on.
E_NO_DELIVERY_POLICY	Application	A continuous application, where the platform document defines no check interval for its blue/green switch.
E_HANDOVER_UNLISTED	Project	A project that the handover ledger lists on neither delivery path.

Table 9: All Constraints, Their Context Classes and What They Reject